

Apex Data Accessor Framework

Introduction:

The main purpose of the Data Accessor Framework is twofold: First, it eliminates duplicate code by having any Query or DML logic for an object exist in one location. This acts as a Model layer in the Model-View-Controller (MVC) design pattern that Salesforce often follows with an Apex Controller being the Controller, and a Lightning Web Component being the View. The second purpose is to allow for easy mocking when unit testing. The purpose of a unit test is to test a single unit of code so that if defects are prevalent, the unit test can be ran to figure out where in the code there may be an issue. Mocking database actions are highly encouraged in unit tests where the particular unit of code under test does not care about what the setup of the data consists of. For example, if there is a method that takes in a list of Accounts and preforms some sort of calculation on a custom field and returns that number, the unit test shouldn't fail because the Account had a new field marked as required where that new field does not interact at all with the method under test. The results in a false negative error in the context of that particular unit test. Mocking can be used to prevent the database action which would cause the DML exception.

Components:

{Object}DAO: The DAO (Data Access Object) is the interface named after the pattern itself. This interface contains all queries related to that particular object.

Example:

```
public interface UserDAO extends DmlInterface {  
    User getUserFromId(String userId);  
    List <User> getUsersFromIdSet( Set <Id> userIds);  
}
```

{Object}DataAccessor:

The DataAccessor is the class that implements the DAO. All of the queries are built within this class.

Example:

```
public inherited sharing class UserDataAccessor extends DmlBase implements UserDAO {  
    public User getUserFromId(String userId) {  
        return [  
            SELECT Id  
                , FirstName  
                , LastName  
                , ContactId  
            FROM User  
            WHERE Id = :userId  
        ];  
    }  
  
    public _List_ <User> getUsersFromIdSet(Set <Id> userIds) {  
        return [  
            SELECT Id  
                , FirstName  
                , LastName  
                , ContactId  
            FROM User  
            WHERE Id IN :userId  
        ];  
    }  
}
```

{Object}DataAccessorMock:

The DataAccessorMock is the class that mocks the queries from its related DataAccessor and is used in unit tests. Each Mock class should have a class variable of type List of the object it is for to represent the data that would be in Salesforce. Then each method should check against that list. For example, if you have a method in an AccountDataAccessor that returns the Account from the passed in AccountId, that mock method should iterate through the class list of mocked Accounts, and if it finds an Account that matches the passed in Id it should return that Account, otherwise a DML exception should be thrown to mimic what would happen if an actual query failed to find an Account.

Example:

```

@Test
public inherited sharing class UserDataAccessorMock extends DmlBaseMock implements UserDao {
    public List <User> mockDatabaseUsers;

    public UserDataAccessorMock() {
        super(new List<User>());
        this.mockDatabaseUsers = (List<User>)super.records;
    }

    public User getUserFromId(String userId) {
        User userToReturn;

        for (User u : mockDatabaseUsers) {
            if (u.Id == userId) {
                userToReturn = u;
            }
        }

        if (userToReturn != null) {
            return userToReturn;
        } else {
            throw new QueryException('User not found');
        }
    }

    public List <User> getUsersFromIdSet(Set <Id> userIds) {
        List <User> usersToReturn = new List<User>();

        for (User u : mockDatabaseUsers) {
            for (Id userId : userIds) {
                if (u.Id == userId) {
                    usersToReturn.add(u);
                }
            }
        }

        return usersToReturn;
    }
}

```

Utility Components:

These components are utilities that provide the DML logic for each object. This code was extracted out to prevent duplicate code within the DataAccessors which likely need DML operations.

DmlInterface: Interface that manages the various methods to preform DML operations.

Code:

```

public interface DmlInterface {
    void insertRecords( List < SObject > newRecords);
    void updateRecords( List < SObject > records);
    void upsertRecords( List < SObject > records);
    void deleteRecords( List < SObject > records);
}

```

DmlBase: Abstract class that contains the implementation of the DML operations. The DataAccessors will extend the DmlBase so that you can call the DML methods using the local instance of the DataAccessor.

Code:

```
public inherited sharing abstract class DmlBase implements DmlInterface {  
    public void insertRecords( List < SObject > newRecords) {  
        insert newRecords;  
    }  
  
    public void updateRecords( List < SObject > records) {  
        update records;  
    }  
  
    public void upsertRecords( List < SObject > records) {  
        upsert records;  
    }  
  
    public void deleteRecords( List < SObject > records) {  
        delete records;  
    }  
}
```

DmlBaseMock: Abstract class that contains the mock implementations of the DML operations. Similar to the DataAccessorMock, the DmlBaseMock uses a class variable to manipulate depending on the DML operation performed.

Code:

```

public abstract class DmlBaseMock implements DmlInterface {
    protected List<SObject> records;
    private static final String ID_FIELD = 'Id'

    public DmlBaseMock(List<SObject> records) {
        this.records = records;
    }

    public List<SObject> getRecords() {
        return this.records;
    }

    public void insertRecords(List<SObject> newRecords) {
        for (SObject record : newRecords) {
            if (record.get(ID_FIELD) != null) {
                throw new DmlException('Cannot insert a record with an ID.');
            }

            this.records.add(record);
        }
    }

    public void updateRecords(List<SObject> records) {
        for (SObject record : records) {
            if (record.get(ID_FIELD) == null) {
                throw new DmlException('Records to update must have a record Id.');
            }

            this.records.add(record);
        }
    }

    public void upsertRecords(List<SObject> records) {
        this.records.addAll(records);
    }

    public void deleteRecords(List<SObject> records) {
        Set<SObject> recordsSet = new Set<SObject>();
        recordsSet.addAll(this.records);

        for (SObject record : records) {
            if (record.get(ID_FIELD) == null) {
                throw new DmlException('Records to delete must have a record Id.');
            }

            if (recordsSet.contains(record)) {
                recordsSet.remove(record);
            }
        }

        this.records = new List<SObject>();
        this.records.addAll(recordsSet);
    }
}

```